```
class X
{
private:
    int* data;
public:
    X():
        data(new int[1000000])
    {}
    ~X()
    {
        delete [] data;
    }
    X(const X& other):                    <-1
        data(new int[1000000])
    {
        std::copy(other.data,other.data+1000000,data);
    }
    X(X&& other):                         <-2
        data(other.data)
    {
        other.data=nullptr;
    }
};
```

The *copy constructor* ❶ is defined just as you might expect: allocate a new block of memory and copy the data across. However, you also have a new constructor that takes the old value by rvalue reference ❷. This is the *move constructor*. In this case you just copy the *pointer to* the data and leave the `other` instance with a null pointer, saving yourself a huge chunk of memory and time when creating variables from rvalues.

For class `X` the move constructor is just an optimization, but in some cases it makes sense to provide a move constructor even when it doesn't make sense to provide a copy constructor. For example, the whole point of `std::unique_ptr<>` is that each non-null instance is the one and only pointer to its object, so a copy constructor makes no sense. However, a move constructor allows ownership of the pointer to be transferred between instances and permits `std::unique_ptr<>` to be used as a function return value—the pointer is *moved* rather than *copied*.

If you wish to explicitly move from a named object that you know you'll no longer use, you can cast it to an rvalue either by using